

Adaptive Queue length Algorithm

Parameters :

- Current Queue Length (QL)
 - Packet Loss Ratio (PLR)
 - Neighbour/Downstream CH Capacity (NC)
 - PLR Threshold (PLR_th)
-

Idea

1. The algorithm only runs when the packet loss ratio becomes too high ($PLR \geq PLR_th$).
 2. When triggered, the CH adjusts its queue length carefully, considering:
 - Its own congestion level (based on PLR)
 - The capacity of its forwarding neighbour (NC) so that it does not overload it
 3. The queue size should not increase too much suddenly (to avoid storing packets unnecessarily) and should not decrease too much (to avoid extra losses).
 4. After calculation, the CH updates its new queue length QL_new .
-

Algorithm :

Step 1: Check if PLR is high

If $PLR < PLR_th$ then,
→ Does nothing, queue size stays normal.

If $PLR \geq PLR_th$ then,
→ Starts queue adaptation.

Step 2: Estimate how serious the congestion is

If PLR is slightly above the threshold → small adjustment

If PLR is much higher → larger adjustment

Step 3: Check neighbour capacity before expanding queue

- If the neighbour is heavily loaded or near full → **avoid increasing queue size too much**
 - If the neighbour has free capacity → **safe to expand queue size slightly**
-

Step 4: Compute a safe new queue size

- If congestion is high and neighbour has low load → increase queue length moderately
 - If congestion is high but neighbour is overloaded → avoid expansion, maybe slightly reduce queue
 - If conditions are moderate → keep queue size stable or adjust slightly
-

Step 5: Assign the new queue length

Update queue buffer to **QL_new** (never below a minimum safe size, never exceeding a max safe limit).

Step 6: Continue normal fuzzy congestion detection

This algorithm works **independently** from fuzzy logic.

Pseudocode :

INPUT:

QL	current queue length
PLR	packet loss ratio
NC	neighbour CH capacity status (LOW, MEDIUM, HIGH)
PLR_th	PLR threshold to trigger adaptation

OUTPUT:

QL_new new adaptive queue length

BEGIN

Step 1: Check if adaptation is needed

```
IF PLR < PLR_th THEN
    QL_new = QL
    RETURN QL_new
ENDIF
```

Step 2: Determine congestion severity

```
IF PLR is slightly above PLR_th THEN
    congestion_level = "MILD"
ELSE IF PLR is moderately above PLR_th THEN
    congestion_level = "MODERATE"
ELSE
    congestion_level = "SEVERE"
ENDIF
```

Step 3: Adjust based on neighbour capacity

```
IF NC == "HIGH" THEN
    // neighbour can handle more traffic
    IF congestion_level == "MILD" THEN
        QL_new = QL + small_increase
    ELSE IF congestion_level == "MODERATE" THEN
        QL_new = QL + moderate_increase
    ELSE
        QL_new = QL + large_increase
    ENDIF
ELSE IF NC == "MEDIUM" THEN
    // avoid big changes
    IF congestion_level == "SEVERE" THEN
        QL_new = QL + small_increase
    ELSE
        QL_new = QL    // keep it stable
    ENDIF
```

```
ELSE IF NC == "LOW" THEN
    // neighbour is overloaded, avoid expansion
    IF congestion_level == "SEVERE" THEN
        QL_new = QL - small_decrease
    ELSE
        QL_new = QL    // no safe expansion
    ENDIF
ENDIF
ENDIF
```

Step 4: Ensure QL_new stays within safe limits
QL_new = clamp(QL_new, MIN_LIMIT, MAX_LIMIT)

```
RETURN QL_new
```

```
END
```